

Fault Aware Training For Spiking Neural Networks

Pin-Yu Wu
National Taiwan University
Taipei, Taiwan
b11502041@ntu.edu.tw

Yu-Shiang Wang
National Taiwan University
Taipei, Taiwan
b11504024@ntu.edu.tw

Abstract—Spiking Neural Networks (SNNs) map naturally onto neuromorphic hardware, but that hardware is vulnerable to silent neuron and synapse faults. We adapt Fault Aware Training (FAT) to SNNs: hidden-layer faults are injected in the training forward pass and a fresh fault list is drawn every mini-batch, so healthy units learn to cover for broken ones. Training, validation and test share the same $T = 25$ rate-coded path. On MNIST (10 seeds, hidden-layer hard-to-spike faults, $p = 0.25$) FAT matches fault-free accuracy of ordinary back-propagation (97.90% vs. 97.72%) and raises faulty-chip accuracy from $87.53\% \pm 2.48$ to $96.91\% \pm 0.21$ (+9.38 pp, paired t -test $p = 6.8 \times 10^{-7}$). On Fashion-MNIST (3 seeds) the faulty-chip gain is +19.8 pp. Ablations show that *per-batch resampling* is the ingredient that matters; restoring the incoming weights of faulty units after the optimiser step is optional and does not explain the gain. The recipe is fault-model specific: a network trained under hard-to-spike faults does not survive saturated neurons, and vice versa. A training-free triple vote (TMR) helps ordinary BP but stays below a single FAT copy and costs $3\times$ inference.

Index Terms—Spiking Neural Networks, Fault Tolerance, Neuromorphic Computation, Robust Training, Hardware Faults.

I. INTRODUCTION

Spiking Neural Networks process information with discrete spikes and are a natural fit for neuromorphic chips [4], [5]. Once a network is mapped onto silicon, manufacturing defects, ageing and environmental noise can silence neurons, lock them on, or freeze synaptic weights. Classical hardware remedies—replication, ECC—spend the energy SNNs are meant to save.

Software-level fault tolerance trains the network so that it keeps working when some units fail. Zahid *et al.* [1] injected simulated hardware faults into conventional (non-spiking) networks during training. We carry that idea to an SNN and pin down *which* part of the recipe is responsible for the gain. Three findings follow.

- 1) A shared rate-coded train/test path plus FAT restores almost all of the accuracy that hidden-layer hard-to-spike faults remove, without hurting a healthy chip (Table I).
- 2) Drawing a new fault list every mini-batch is necessary. Freezing one fault list for the whole run is worse than ordinary back-propagation. An optional weight-recovery step that rolls back the incoming weights of units that were broken in that batch does not change the result (Table II).
- 3) The gain does not transfer across fault *types*. Training under hard-to-spike faults leaves the network as fragile as the baseline when neurons saturate, and training under saturation is what repairs saturation.

Code that reproduces every table is released with the paper.

II. RELATED WORK

SNN training uses a surrogate gradient at the non-differentiable spike [2], [3]; we use the arctan surrogate in snntorch and clip the gradient ℓ_2 norm to 10. Neuromorphic processors such as Loihi [5] make the energy argument concrete, and surveys place SNNs as a hardware-aware alternative to deep networks [4].

On the fault-tolerance side, Spyrou *et al.* [6] analyse how individual neuron faults propagate in SNNs. ReSpawn [7] mitigates unreliable memories with an energy-aware mapping. Those works change the hardware or the mapper. FAT changes only the training loop: it is complementary, not a competitor to a memory encoder. We therefore compare against (i) ordinary back-propagation on the same SNN and (ii) a training-free triple modular redundancy (TMR) vote on independently faulted copies of that baseline—a classical hardware-redundancy lower bound that costs $3\times$ inference, not a reimplement of ReSpawn.

Zahid *et al.* [1] is the closest training recipe. The differences here are the SNN (LIF, rate coding, surrogate gradient), the requirement that train and test see the same spike-generation process, and the ablation that isolates resampling from weight recovery.

III. PROPOSED TECHNIQUE

The network is fully connected, $784 \rightarrow 256 \rightarrow 32 \rightarrow 10$, with no biases. Each linear map is followed by a leaky integrate-and-fire (LIF) neuron (snntorch `Leaky`, $\beta = 0.9$, threshold 0.5, subtract reset). MNIST pixels in $[0, 1]$ are treated as spike probabilities: at each of $T = 25$ time steps a Bernoulli spike train is drawn and pushed through the three stages; output spikes are summed and used as logits. **Training, validation and test share this forward path.**

A. Fault simulation

Every *hidden* LIF neuron is declared faulty independently with probability p (default 0.25). The class (output) layer is left intact: at $p = 0.25$ a ten-way readout would otherwise lose two or three digits at random and dominate the error. Three models are supported:

- **Hard-to-spike (hts)** — the neuron never emits a spike.
- **Saturated (sat)** — the neuron spikes at every time step.

TABLE I
TEST ACCURACY (%) OF NORMAL BACK-PROPAGATION VS. FAT UNDER
HIDDEN-LAYER HARD-TO-SPIKE FAULTS, $p = 0.25$, $R = 10$ FULL-SET
DRAWS PER SEED. MEAN $\pm$ STD IS OVER THE 10 SEEDS.

seed	N FF	N faulty	FAT FF	FAT faulty
0	97.77	84.54	98.01	96.92
1	97.78	90.61	97.90	97.17
2	97.88	87.01	97.99	96.90
3	97.68	90.02	98.08	96.97
4	97.72	90.00	97.91	96.98
5	97.69	85.47	97.67	96.43
6	97.61	83.38	97.84	97.06
7	97.82	89.13	97.89	96.69
8	97.47	87.96	97.85	97.05
9	97.75	87.18	97.85	96.89
mean $\pm$ std	97.72 $\pm$ 0.12	87.53 $\pm$ 2.48	97.90 $\pm$ 0.11	96.91 $\pm$ 0.21

- **Stuck-at weight (swf)** — one random incoming synapse is frozen at a value drawn uniformly from $[-10, 10]$.

Unless disabled, a new list is drawn every mini-batch so the network cannot specialise to one broken subset. Table I uses hts.

B. Weight recovery

After the Adam step, the incoming weight rows of the neurons that were faulty in that batch can be written back to their pre-step values, so those units do not learn from a batch in which they were broken. The step is on by default for continuity with Zahid *et al.*, but Section IV-B shows that turning it off does not remove the FAT gain. We therefore treat recovery as optional regularisation, not as the mechanism.

C. Evaluation

Two checkpoints are written each run: the best fault-free validation accuracy and the best *faulty* validation accuracy. FAT early-stops and reports the latter; the normal-BP baseline reports the former. Test numbers use the full 10 000-image set. Fault-free accuracy is repeated over R independent rate-coding draws. Faulty accuracy draws R independent fault lists; each list is applied to the *whole* test set (one draw = one simulated chip). We report mean $\pm$ standard deviation over seeds or over the R draws as noted.

IV. EXPERIMENTS

Both methods use Adam (3×10^{-4} , weight decay 10^{-5}), batch size 64, an 8:2 train/validation split, early stopping with patience 5, and seed-controlled data, spikes and fault draws. Table I is `run_table1.py` with 10 seeds, 20 epochs and $R = 10$ (hts, output layer healthy).

A. Main result

Under hidden-layer hts faults FAT keeps $96.91\% \pm 0.21$ versus $87.53\% \pm 2.48$ for normal BP (+9.38 pp, paired $t = 12.18$, $p = 6.8 \times 10^{-7}$). Fault-free accuracy is essentially unchanged (+0.18 pp, $p = 0.002$). Every seed shows a positive faulty-chip gap (Fig. 1). FAT also shrinks the seed-to-seed spread: the faulty standard deviation falls from 2.48 to 0.21 points.

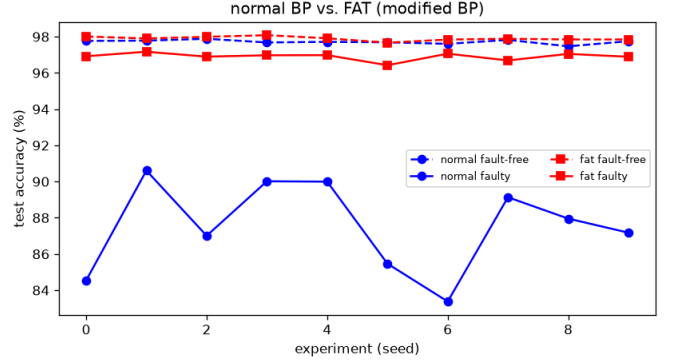


Fig. 1. Per-seed test accuracy for the run in Table I. FAT (red) tracks the fault-free curve even when 25% of hidden neurons are silenced; normal BP (blue) does not.

B. Ablations and cross-fault tests

Table II and Fig. 2 train six recipes for three seeds and test every model under hts, sat and swf ($p = 0.25$, $R = 10$). Three observations follow.

a) *Resampling, not recovery.*: Full FAT (hts, resample on, recovery on) reaches $97.02\% \pm 0.14$ under hts. Disabling recovery leaves $96.84\% \pm 0.19$. Disabling resampling— one fault list for the whole run—drops hts accuracy to $82.19\% \pm 1.41$, below the $86.88\% \pm 0.32$ of ordinary BP. A network that only ever sees one broken subset overfits that subset and generalises worse to a new chip than a network that never saw faults.

b) *Train and test must match.*: hts-trained FAT is as weak as the baseline on saturated neurons ($\approx 34\%$ vs. $\approx 36\%$). Training under saturation is what repairs saturation ($94.31\% \pm 0.56$). Stuck-at-weight training repairs swf (97.14%) and transfers partly to hts (92.40%), but not to sat. FAT is not a universal robustness potion; it is exposure to the fault the chip will actually have.

c) *Healthy-chip cost.*: All hts variants keep fault-free accuracy at $97.6\text{--}98.0\%$. sat training is the only recipe that pays a visible clean-chip tax (96.57%), consistent with forcing many neurons to spike every step.

C. Accuracy versus fault rate

Evaluating the seed-0 normal and FAT checkpoints at $p \in \{0.05, 0.10, 0.25, 0.50\}$ (hts, sat, swf; $R = 10$) produces Fig. 3. The hts gap widens as p grows: FAT stays near the clean-chip line while normal BP falls. The sat curves of both models drop together unless the trainer was sat (Table II), which is the same mismatch already visible at $p = 0.25$.

D. Redundancy baseline

TMR runs three independently faulted copies of a frozen checkpoint and majority-votes the class. It is a training-free, $3\times$ -compute reference, not an SNN-specific mapper. On seeds 0–2 (hts, $p = 0.25$, $R = 10$) TMR lifts normal BP from 87.39% (single copy) to $93.58\% \pm 2.07$, still below a single FAT copy on the same seeds (97.00%). Applying TMR on top of FAT adds almost nothing ($97.55\% \pm 0.04$). The comparison

TABLE II
ABLATIONS (3 SEEDS). EACH ROW IS A TRAINING RECIPE; COLUMNS ARE FAULT-FREE ACCURACY AND TEST ACCURACY (%) UNDER EACH FAULT MODEL AT $p = 0.25$. MEAN $\pm$ STD OVER SEEDS.

train	FF	hts	sat	swf
normal BP	97.82 $\pm$ 0.06	86.88 $\pm$ 0.32	35.63 $\pm$ 1.81	94.91 $\pm$ 0.98
FAT (hts)	97.97 $\pm$ 0.04	97.02 $\pm$ 0.14	34.40 $\pm$ 4.75	96.57 $\pm$ 0.25
hts, no recovery	97.89 $\pm$ 0.16	96.84 $\pm$ 0.19	36.17 $\pm$ 4.22	96.50 $\pm$ 0.46
hts, no resample	97.58 $\pm$ 0.12	82.19 $\pm$ 1.41	32.46 $\pm$ 4.34	93.20 $\pm$ 0.50
FAT (sat)	96.57 $\pm$ 0.15	77.49 $\pm$ 3.46	94.31 $\pm$ 0.56	92.48 $\pm$ 1.61
FAT (swf)	97.75 $\pm$ 0.20	92.40 $\pm$ 0.65	58.00 $\pm$ 7.40	97.14 $\pm$ 0.29

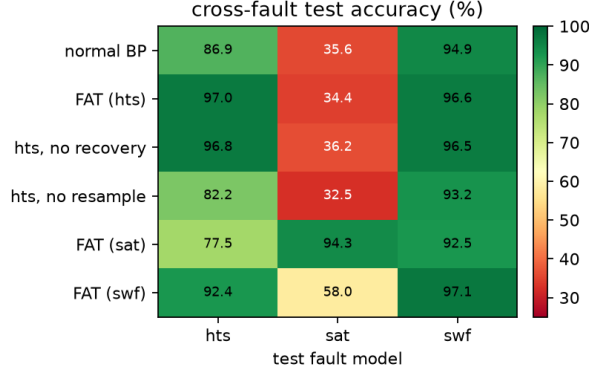


Fig. 2. Mean cross-fault test accuracy (%). The diagonal (train-on-X / test-on-X) is high; off-diagonal cells, especially anything tested under saturation, collapse unless the trainer saw saturation.

TABLE III
FASHION-MNIST, SAME PROTOCOL AS TABLE I (3 SEEDS, HTS, $p = 0.25$, $R = 10$).

seed	N FF	N faulty	FAT FF	FAT faulty
0	87.56	65.19	86.21	86.02
1	88.12	67.96	85.39	86.87
2	87.66	66.62	87.36	86.15
mean $\pm$ std	87.78 $\pm$ 0.30	66.59 $\pm$ 1.39	86.32 $\pm$ 0.99	86.35 $\pm$ 0.46

is cost: TMR spends three forwards at test time; FAT spends one and moves the work into training.

E. Fashion-MNIST

The same architecture and protocol with 3 seeds on Fashion-MNIST (Table III) gives a larger faulty-chip gap: 66.59% $\pm$ 1.39 (normal) vs. 86.35% $\pm$ 0.46 (FAT), +19.76 pp (paired $t = 34.7$, $p = 8.3 \times 10^{-4}$). Fault-free FAT is slightly lower (−1.46 pp, $p = 0.17$, not significant at 3 seeds). The gain is therefore not MNIST-specific; the harder dataset is also where ordinary BP loses more under hidden-layer faults.

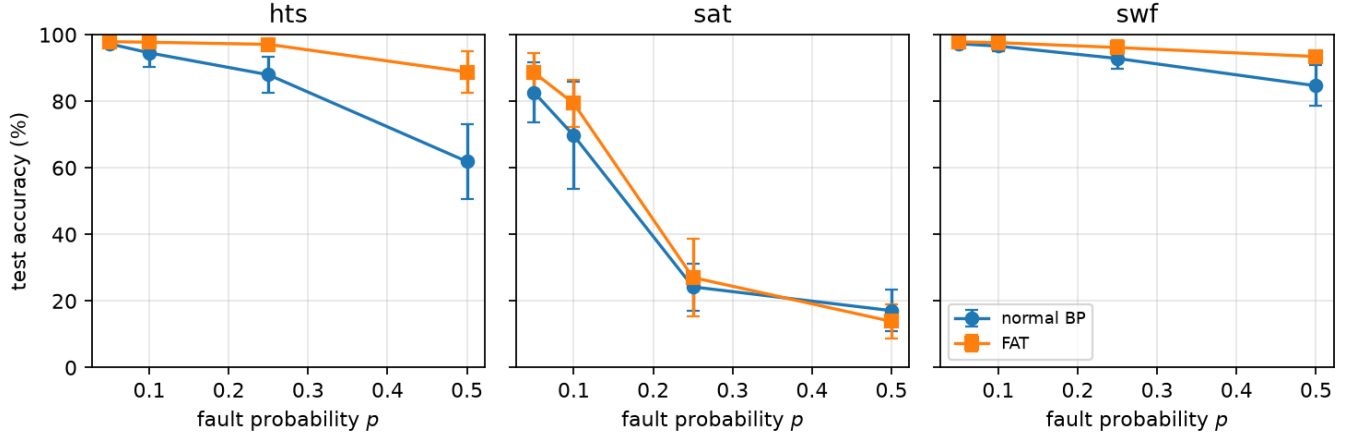


Fig. 3. Test accuracy vs. fault probability for the seed-0 normal-BP and FAT (hts) checkpoints. One panel per test fault model.

V. LIMITATIONS

Faults are drawn independently per hidden neuron. Real silicon defects are spatially correlated, layer-dependent, and include analog stuck-at and timing errors that we do not model. The output layer is protected, which is a modelling choice (a dead class logit cannot be recovered by the others) but is more optimistic than a uniform fault rate over every core. Results are on a three-layer fully connected SNN; a convolutional variant is implemented (`--arch conv`) but not reported here. We do not measure energy or accuracy on a physical neuromorphic chip. TMR is a software vote, not chip-level lockstep.

VI. CONCLUSION

FAT injects hidden-layer hardware faults into a rate-coded SNN forward pass and redraws them every batch. On MNIST and Fashion-MNIST that is enough to recover most of the accuracy lost to hard-to-spike faults. The mechanism is exposure to many fault layouts, not the optional weight-recovery rollback, and the exposure must match the fault the chip will see. A triple vote at test time costs more and still does not beat a single FAT copy. The recipe is a software complement to neuromorphic fault tolerance and is fully reproducible from the accompanying code.

REFERENCES

- [1] U. Zahid, G. Gambardella, N. J. Fraser, M. Blott and K. Vissers, “FAT: Training Neural Networks for Reliable Inference Under Hardware Faults,” in *Proc. IEEE ITC*, 2020, pp. 1–10.
- [2] J. K. Eshraghian *et al.*, “Training Spiking Neural Networks Using Lessons From Deep Learning,” *Proc. IEEE*, vol. 111, no. 9, pp. 1479–1495, Sep. 2023.
- [3] E. O. Neftci, H. Mostafa and F. Zenke, “Surrogate Gradient Learning in Spiking Neural Networks: Bringing the Power of Gradient-Based Optimization to Spiking Neural Networks,” *IEEE Signal Process. Mag.*, vol. 36, no. 6, pp. 51–63, Nov. 2019.
- [4] K. Roy, A. Jaiswal and P. Panda, “Towards spike-based machine intelligence with neuromorphic computing,” *Nature*, vol. 575, pp. 607–617, 2019.
- [5] M. Davies *et al.*, “Loihi: A Neuromorphic Manycore Processor with On-Chip Learning,” *IEEE Micro*, vol. 38, no. 1, pp. 82–99, 2018.
- [6] T. Spyrou *et al.*, “Neuron Fault Tolerance in Spiking Neural Networks,” in *Proc. DATE*, 2021, pp. 743–748.

- [7] R. V. W. Putra, M. A. Hanif and M. Shafique, “ReSpawn: Energy-Efficient Fault-Tolerance for Spiking Neural Networks considering Unreliable Memories,” in *Proc. ICCAD*, 2021, pp. 1–9.